UiT

THE ARCTIC UNIVERSITY OF NORWAY

EXAMINATION PAPER

Home exam

Home exam in:	**FYS-3012/FYS-8012 Pattern Recognition**
Hand-out:	**Tuesday September 29, 2026, 09:00**
Hand-in:	**Tuesday October 13, 2026, 14:00**

The exam contains 14 pages including this cover page

Contact person:	**Kristoffer Wickstrøm**
Email:	**kristoffer.k.wickstrom@uit.no**

UiT
THE ARCTIC
UNIVERSITY
OF NORWAY

Before You Start

Module examination

This is a take-home exam for FYS-3012/FYS-8012 Pattern Recognition. Assessment of this assignment counts 50% towards the final grade in the course. Note that access to the final examination requires submission of this take-home exam. A report which clearly does not constitute an academic attempt will be deemed as not submitted.

Portfolio instructions

Make sure the report explains your reasoning and does not merely provide equations without further explanation. Computer code should be commented in such a way that any person with programming knowledge should be able to understand how the program works. Like your report, the code must be your own individual work.

As there is a lot of code available online on the web, make sure that your report and code clearly show that you understand what you are doing.

You should not use pre-built functions to implement your algorithms. For example, you should not use pre-built algorithms like:

```
from sklearn.naive_bayes import GaussianNB
```

However, you are free to use, and encouraged to use, basic built-in functions from e.g. NumPy and SciPy such as:

```
from scipy.spatial.distance import pdist
import numpy as np
```

You can also use scikit-learn functions to calculate performance metrics like accuracy.

The only exception to the instructions above is in the case of Support Vector Machines, where you are allowed to use the pre-built function in scikit-learn.

UNIVERSITY OF TROMSØ

Exam format

Your report should be no more than *10 pages*. This does not include references (that means you can have as many references as you want, they do not contribute to the page count). Make sure your figures are large enough to be viewable. Report results in tables. Make sure plots are labelled. A clean presentation of your work will make it easier for evaluators to understand and appreciate what you have done.

Hand-in format

Please submit a single `.zip` file that contains two folders, one called `doc` that contains your report, and another one called `src` containing the code. The file name of the `.zip` file should follow the format `homeexam_candidate`*XX*`.zip` (replace *XX* with your candidate number obtained from StudWeb) for anonymity.

Follow the hand-in instruction in WiseFlow and make sure to submit before the WiseFlow room closes.

Problem 1

You are a newly hired machine learning engineer at Pattern Recognition Solutions, a company that offers data-driven problem solving to customers around the world and is known for its high expertise. As a junior engineer, you are eager to prove that you belong, and you look forward to your first assignments.

For your first task, Pattern Recognition Solutions has been contacted by the Norwegian Directorate of Fisheries, which regulates salmon farming along the Norwegian coast. Inspecting every farm site is expensive, so the directorate wants to prioritise which sites to inspect. They have asked you to develop a model that predicts which salmon farm sites are likely to breach the regulatory sea-lice limit in the coming years, based on each site's track record and registry information.

You are provided with data from Norwegian salmon farm sites. The data set is described in detail in **Appendix A**, and is available on Canvas under the "Home exam" module as the file `task1_lice_forecast.npz`.

```
import numpy as np
data = np.load('task1_lice_forecast.npz')                          # load the archive

X_registry          = data['X_registry']                           # facility + geography
X_full              = data['X_full']                               # registry + lice history
y                   = data['y']                                    # breach_next label (0/1)
feature_names_registry= data['feature_names_registry']             # names for X_registry
feature_names_full  = data['feature_names_full']                   # names for X_full
groups              = data['groups']                               # siteNumber, for group-aware splits
anchor_year         = data['anchor_year']                          # anchor year Y of each row
```

`X_registry` contains only the facility and geography features and is used in Problem (1e); `X_full` adds the site's lice history (the `past_` columns) and is used in Problem (1f). Both matrices have one row per site–year, in the same row order as `y`.

The data contain weekly sea-lice observations summarised per site and year, together with facility characteristics and geographic information. Each row corresponds to one farm site at one anchor year, and the target/label `y` (`breach_next`) indicates whether the site will breach the regulatory sea-lice limit in the following two years. You will approach the task in two steps: first using only the registry information about the sites (Problem (1e)), then also using each site's lice history (Problem (1f)). In both steps you should compare the three classifiers from Problems (1a)–(1c), in Problem (1h) also a bagged version of the naive Bayes classifier, and relate the results to the dummy baseline from Problem (1d).

(1a) Describe the key ideas and equations of the naive Bayes classifier using Parzen-window density estimation for the class-conditional densities. Briefly explain the details and hyperparameters you need to choose for your implementation, and how the estimated densities are combined with the class priors to classify a new feature vector.

(1b) Describe the key ideas and equations of the sum of least squares classifier. Briefly explain the details and hyperparameters that you need to decide for your implementation.

(1c) Describe the key ideas and equations of the multilayer perceptron. Briefly explain the details and hyperparameters that you need to choose for your implementation.

(1d) When working with real world data performance is never perfect, and it can be difficult to know the value of your algorithm. Design your own dummy classifier that you can use as a baseline to determine if your algorithms in (1a)–(1c) give meaningful performance.

(1e) We first ask how much information there is in the registry alone. Using the registry data (`X_registry`), train the three classifiers from Problems (1a)–(1c) to predict `breach_next`, and evaluate and compare their performance in terms of precision, recall, and F1 score. Also evaluate the performance of your dummy classifier from (1d). Did the classifiers actually achieve a performance increase over the baseline? Discuss.

(1f) Repeat Problem (1e), but now with the full set of features (`X_full`). How does the classifier perform now? Discuss.

(1g) Briefly explain the idea of *bagging* (bootstrap aggregating): how the base classifiers are trained and how their outputs are combined for classification. Explain how bagging could be applied to the naive Bayes classifier with Parzen windows from Problem (1a).

(1h) Repeat Problem (1f), but replace the naive Bayes + Parzen classifier with the bagged version you described in Problem (1g). Compare its performance with the unbagged version and with the other two classifiers and discuss the results.

Problem 2

For your second task, Pattern Recognition Solutions has been contacted by the Norwegian Food Safety Authority (Mattilsynet, https://www.mattilsynet.no/en), which wants help checking its water-quality reports. The authority has given your team two sets of measurements from Norwegian water supply systems. The first, `X_reference`, is a large set that the authority considers representative of normal operation. The second, `X_query`, is a smaller batch of systems that the authority is less sure about: they suspect that some of the reports are unreliable, but they do not know which ones or why. Your task is to learn the normal structure of the reference data with principal component analysis, and use it to find and characterise the records in the query set that do not fit.

Both sets are provided in the file `task2_water_reports.npz`, available on Canvas under the "Home exam" module. Load it as follows:

```
import numpy as np
data = np.load('task2_water_reports.npz')                    # load the archive

X_reference          = data['X_reference']                   # learn normal structure
feature_names_reference = data['feature_names_reference']    # column names
X_query              = data['X_query']                       # records to inspect
feature_names_query    = data['feature_names_query']         # column names
record_ids_query       = data['record_ids_query']            # ids, for reporting
```

Both matrices have the same eight columns. The reference and query sets contain 915 and 246 systems, respectively, and the systems in the two sets are distinct. Appendix B describes the data and the preprocessing rules. Fit every preprocessing parameter and the PCA model using `X_reference` only; use `X_query` only for the final inspection.

(2a) Describe the key ideas and equations of PCA. Explain how the principal components are chosen and how the fraction of explained variance is calculated. Give the equations for projecting a feature vector onto the first q components and reconstructing it in the eight-dimensional, transformed and standardised feature space. Explain why this reconstruction is generally an approximation when q is smaller than the number of features.

(2b) Implement PCA and fit it to the prepared reference data. Retain the first two components. Report their individual and cumulative explained variance, and plot both data sets in this two-dimensional space. Report the three features with the largest absolute coefficients in the first component and briefly interpret what this component represents.

(2c) Propose an anomaly score based on the PCA reconstruction. Start from the reconstruction residual $r = x - \hat{x}$, and standardise each feature's residual using the mean and standard deviation of the reference residuals for that feature, so that residuals of features with different natural variability become comparable. Define your per-record score and explain what it measures.

(2d) Implement your score on the prepared data. Choose the detection threshold using the reference set only, for example a high percentile of the reference scores, and explain your choice. Apply the threshold to the query set, report the number of flagged records, and list the ten highest-scoring records by their `record_ids`.

Appendix A: Data description

Background

Norway is the world's largest producer of farmed Atlantic salmon. The fish are kept in large net pens (*cages*) that float in the sea, and each farm operates one or more such sites along the coast. Norwegian waters are divided into 13 *production areas* used for management and regulation.

Salmon in sea cages are exposed to *sea lice*: small parasites that attach to the fish and feed on them. High lice levels are bad for fish welfare, and lice from farms can also spread to wild salmon, so the authorities regulate them strictly. Every farm must count the lice on its fish each week and report the numbers to a public database. The regulations set a maximum allowed number of adult female sea lice per fish (0.5 in most production areas, 0.2 in some); a site that reports more than this is *above the lice limit*. To get rid of lice, a farm may also *fallow* a site: keep it empty of fish for a period. No lice are counted while a site is fallow.

Each farm site also has a *certificate* in a public registry (the NYTEK register), which describes the technical installation: how many cages, rafts and mooring systems it has, how many production units it is licensed for, and any deviations (*nonconformities*) found at inspection.

What the data set contains

The data set behind `task1_lice_forecast.npz` contains 3,116 rows covering 496 salmon farm sites. Each row is one *site at one* ***anchor year*** Y (anchor years 2018–2024), and contains three kinds of information:

- **Lice history:** summary of the site's lice situation over the *past* three years ($Y-2$ to Y).
- **Facility and geography:** characteristics of the site from the certificate registry, plus its location and production area.
- **The future:** the same lice summaries computed over the *following* two years ($Y+1$ to $Y+2$), and the target label derived from them.

A row is only included if the site reported lice counts in at least 10 eligible weeks (weeks where the site was fallow are not eligible) both in the past window and in the future window.

The input features (`X`)

The archive contains two feature matrices with the same 3,116 rows, one row per site–year:

- `X_registry` — shape $(3116, 28)$: the 28 facility and geography features, i.e. what the registry knows without the site's lice record. Column names in `feature_names_registry`.
- `X_full` — shape $(3116, 31)$: the same 28 columns followed by the 3 lice-history features. This is [`X_registry` | lice history]. Column names in `feature_names_full`.

Row i belongs to site `groups[i]` in anchor year `anchor_year[i]`. The table below lists every column of

`X_full` by its index (the same index also works on `X_registry` for indices 0–27). The *Range* column is the minimum–maximum over all 3,116 rows, and the *Example* column gives the value of the worked example below.

#	Feature	Description	Range	Ex.
0	`productionUnitsMax`	Number of production units the site is licensed for (its size).	1–24	8
1	`productionUnitsPresent`	Number of production units currently in use.	0–15†	2
2	`n_conformity_items`	Number of requirement items checked at the site's latest inspection.	0–30	3
3	`n_open_nonconformities`	Number of unresolved deviations found at inspection.	0–30	0
4	`worst_nonconformity_rank`	Severity of the worst deviation: 0 = none, 3 = major.	$\{0, 3\}$	3
5	`n_rafts`	Number of rafts at the site.	0–5	0
6	`n_enclosures`	Number of cages (enclosures) at the site.	0–13	1
7	`n_floaters`	Number of floaters at the site.	0–5	1
8	`n_mooring_systems`	Number of mooring systems at the site.	1–4	1
9	`n_integrated`	Number of integrated components.	0–1	0
10	`n_aux`	Number of auxiliary equipment components.	0–45	1
11	`any_enclosure_closed`	1 if the site has any closed or semi-closed cage.	$\{0, 1\}$	0
12	`any_floater_submersible`	1 if the site has any submersible (lowerable) floater.	$\{0, 1\}$	0
13	`lat`	Latitude of the site (degrees north).	58.2–71.0	62.74
14	`lon`	Longitude of the site (degrees east).	4.7–30.4	7.88
15	`area_Andøya til Senja`	One-hot indicator for production area "Andøya til Senja".	$\{0, 1\}$	0
16	`area_Helgeland til Bodø`	One-hot indicator for production area "Helgeland til Bodø".	$\{0, 1\}$	0

#	Feature	Description	Range	Ex.
17	area_Karmøy til Sotra	One-hot indicator for production area "Karmøy til Sotra".	$\{0, 1\}$	0
18	area_Kvaløya til Loppa	One-hot indicator for production area "Kvaløya til Loppa".	$\{0, 1\}$	0
19	area_Nord-Trøndelag med Bindal	One-hot indicator for production area "Nord-Trøndelag med Bindal".	$\{0, 1\}$	0
20	area_Nordhordaland til Stadt	One-hot indicator for production area "Nordhordaland til Stadt".	$\{0, 1\}$	0
21	area_Nordmøre og Sør-Trøndelag	One-hot indicator for production area "Nordmøre og Sør-Trøndelag".	$\{0, 1\}$	0
22	area_Ryfylke	One-hot indicator for production area "Ryfylke".	$\{0, 1\}$	0
23	area_Stadt til Hustadvika	One-hot indicator for production area "Stadt til Hustadvika".	$\{0, 1\}$	1
24	area_Svenskegrensen til Jæren	One-hot indicator for production area "Svenskegrensen til Jæren".	$\{0, 1\}$	0
25	area_Vest-Finnmark	One-hot indicator for production area "Vest-Finnmark".	$\{0, 1\}$	0
26	area_Vestfjorden og Vesterålen	One-hot indicator for production area "Vestfjorden og Vesterålen".	$\{0, 1\}$	0
27	area_Øst-Finnmark	One-hot indicator for production area "Øst-Finnmark".	$\{0, 1\}$	0
28	past_mean_af	Average number of adult female sea lice per fish over the past window ($Y - 2$ to Y).	0–3.85	0.326
29	past_frac_above	Fraction of eligible weeks in the past window with lice above the regulatory limit.	0–0.60	0.227
30	past_n_weeks	Number of eligible weeks (reported, non-fallow) in the past window.	10–154	119

Indices 0–27 form `X_registry`; indices 28–30 are appended in `X_full`. Exactly one of the 13 `area_*` indicators (indices 15–27) is 1 in each row. †`productionUnitsPresent` is the only column with missing values (see below). The organisation number of the license holder (`licenseHolderOrgNr`) is an identifier and is *not* included in either matrix.

The label (y)

The label is `breach_next`, a binary variable taking values 0 or 1:

$$y_i = 1 \quad \Longleftrightarrow \quad \texttt{fut_frac_above}_i \geq 0.05.$$

That is, the site is labelled a *breacher* ($y = 1$) if it was above the regulatory lice limit in at least 5% of its eligible weeks during the following two years ($Y + 1$ to $Y + 2$). Of the 3,116 rows, 589 are breachers and 2,527 are not, so the positive rate is about 19% and the classes are imbalanced. This is why a dummy classifier is a useful reference (Problem (1d)).

Leakage warning. The label is derived from the site's lice situation in the following two years. No future-window values are included in `X_registry` or `X_full`, so the label cannot be reconstructed from the input features; be careful not to reintroduce any future information when you build your own features.

A worked example

The following snippet prints one row—site 12884 in anchor year 2018—with its feature names and label:

```
i = np.where((groups == 12884) & (anchor_year == 2018))[0][0]                    # find the row

for j, name in enumerate(feature_names_full):                                    # print feature values
    print(f'{j:2d}  {name:24s} {X_full[i, j]:g}')
print('y =', y[i])                                                               # the label
```

The *Example* column of the table above shows the values. Reading the row:

- **Registry (0–27):** the site is licensed for `productionUnitsMax` = 8 production units but has `productionUnitsPresent` = 2 in use. Its latest inspection checked `n_conformity_items` = 3 items and found a *major* deviation (`worst_nonconformity_rank` = 3), but no open deviations (`n_open_nonconformities` = 0). It has 1 cage, 1 floater and 1 mooring system. Geographically it sits at 62.74°N, 7.88°E, in the production area "Stadt til Hustadvika" (`area_Stadt til Hustadvika` = 1, all other `area_*` columns = 0).
- **Lice history (28–30, only in `X_full`):** over the past three years the site averaged `past_mean_af` = 0.326 adult female lice per fish, and was above the limit in `past_frac_above` = 22.7% of its 119 eligible weeks.
- **Label:** in the following two years the site was above the limit in 29.8% of its eligible weeks. Since $0.298 \geq 0.05$, the site is labelled a breacher ($y = 1$). This future-window quantity is shown only to explain the label — it is not an input feature.

Loading the data

The file is a plain NumPy archive and can be inspected and loaded with:

```
import numpy as np
data = np.load('task1_lice_forecast.npz')                                        # load the archive
```

```
X_registry           = data['X_registry']                    # facility + geography
X_full               = data['X_full']                      # registry + lice history
y                    = data['y']                         # breach_next label (0/1)
feature_names_registry= data['feature_names_registry']       # names for X_registry
feature_names_full    = data['feature_names_full']             # names for X_full
groups               = data['groups']              # siteNumber, for group-aware splits
anchor_year          = data['anchor_year']                 # anchor year Y of each row
```

The columns of both matrices follow the numbering in the feature table: indices 0–27 are the facility and geography features (with the production area one-hot encoded as `area_*`), and indices 28–30 — present only in `X_full` — are the lice-history features. Every column is already numeric, so the matrices can be passed to a classifier directly, except that you must decide how to handle the missing values in `productionUnitsPresent`.

Missing values and caveats

- `productionUnitsPresent` is missing in about 11% of the rows (stored as `NaN` in both matrices); all other columns are complete. You should handle this missingness in your pipeline.
- The facility variables come from the site's *current* certificate, not the certificate valid at the anchor year. For historical rows this is a slight mismatch, which is acceptable for this exercise.
- The file contains several rows from the same site (one per anchor year), listed in `groups`. Keep this in mind when you split the data into training and test sets — a plain random split leaks information across anchor years of the same site.

Source

The data were compiled from two public Norwegian sources: weekly salmon-lice reports from the BarentsWatch Fishhealth service (https://www.barentswatch.no/), and site certificates from the Norwegian Directorate of Fisheries' NYTEK register (https://www.fiskeridir.no/). The data set was prepared for this exam and may be used freely within the course.

Appendix B: Water-quality reporting data

The data originate from the Norwegian Food Safety Authority (Mattilsynet), which publishes reported measurements from water supply systems: https://www.mattilsynet.no/drikkevannsforsyning/opplysninger-om-vannforsyningssystemer/vannforsyningssystemer-til-lands. This exercise uses the distributed-water analysis table from that collection.

Each row represents one water supply system in 2024. The eight numerical features are annual medians. The table below lists every column of the feature matrices by its index; the columns follow the order in this table and in `feature_names_reference` and `feature_names_query`.

#	Feature	Description	Reported unit
0	`conductivity`	Electrical conductivity.	mS/m
1	`sodium`	Sodium.	mg Na/L
2	`chloride`	Chloride.	mg Cl/L
3	`sulfate`	Sulfate.	mg SO_4/L
4	`ph`	pH.	pH scale
5	`color`	Water colour.	mg Pt/L
6	`turbidity`	Turbidity.	FNU
7	`toc`	Total organic carbon.	mg C/L

The measurements are supplied in their reported units; preprocessing has not yet been applied.

The reference and query sets

Only systems with all eight features available were retained. After cleaning, 1,161 systems remain, split into two non-overlapping sets:

- `X_reference`, shape $(915, 8)$: systems the authority considers representative of normal operation. Use this set to fit the preprocessing and the PCA model.
- `X_query`, shape $(246, 8)$: systems under review. Use this set only for the final inspection; do not use it to fit any part of the model.

The two sets come from the same reporting system in the same year, so they are directly comparable. No information about individual query records is provided with the data.

Preprocessing and numerical conventions

Apply the following steps before fitting PCA:

1. Transform every feature except `ph` using $\log(1 + x)$, where log is the natural logarithm. In NumPy, use `np.log1p`.

2. Standardise all eight resulting features by subtracting their reference-set means and dividing by their reference-set standard deviations. Calculate these standard deviations with divisor n (`ddof=0`).

3. Apply the same transformation and the same reference means and standard deviations to the query set. Do not standardise the two sets independently.

Use the usual centred PCA with orthonormal component vectors, retaining two components. Do not whiten the component coordinates. Projections, reconstructions and residuals must be calculated in the transformed, standardised eight-dimensional space, and never by comparing a reconstructed standardised value with an untransformed measurement.

If you choose a percentile threshold, use `np.quantile(scores, 0.95)` with its default linear interpolation, computed on the reference scores. A score equal to the threshold is not flagged.

Identifiers

The archive also contains `record_ids_reference` and `record_ids_query`. These are pseudonymous identifiers in the same row order as their corresponding feature matrices. Use them to refer to individual records, not as model features. No class labels are provided for either set.

Source

The units are documented in Mattilsynet's analysis data dictionary: https://data.mattilsynet.no/vannverk/Datafelt%20-%20Vannforsyningssystem_analyser.pdf.

References